

Assignment 3: Build, Load, Measure, and Explain an Event-Driven Server

Linux Systems and Performance - Week 3

epoll Server -> FD-Based Event Loop -> Worker Offload -> Worker Pool & Backpressure

1. Assignment Philosophy

This assignment is not evaluated only by whether the server accepts connections. You will build one server architecture, apply controlled load, observe responsiveness and resource behavior, measure how architectural changes affect the system, and explain the causes of the results.

BUILD -> LOAD -> MEASURE -> ANALYZE -> EXPLAIN

Important: All coding tools may be used. However, you are responsible for understanding, debugging, modifying, and explaining all submitted code and experimental results. Implementation alone is not sufficient for full credit.

2. Challenge Levels

Level	Build	Load / Trigger	Measure / Observe	Main Question
Level 1	single-thread nonblocking epoll server	multiple clients + partial requests	correctness + responsiveness	How can one event-loop thread serve many clients?
Level 2	timerfd + eventfd + one worker	light requests during slow compute	blocking vs. offload latency	Why can slow work still block a nonblocking server?
Level 3	worker pool + bounded queues	increasing concurrency / overload	throughput, latency, queueing, perf	Where does scaling stop, and what happens under overload?

Levels are cumulative. Use the same line-oriented protocol and the instructor-provided load generator across levels so that architectural comparisons are meaningful. Absolute performance across different machines is not graded.

3. Common Interface and Rules

- Implement the required server in C. Use one evolving server codebase or clearly related versions so architectural changes can be compared.
- Use IPv4 TCP and a line-oriented protocol. Each request and response ends with '\n'. Required request/response size may be limited to 256 bytes.
- The required implementation uses level-triggered epoll. EPOLLET/EPOLLONESHOT are optional extensions.
- Do not create one process or one thread per client in the required epoll implementation.
- All client sockets used by the event loop must be nonblocking.
- Use the instructor-provided loadgen for required measurements. Do not replace it with an unrelated benchmark unless explicitly approved.
- For every performance comparison, keep machine, build options, workload definition, and request mix fixed. Repeat each configuration at least three times and report the median.
- Record exact commands and configuration values. Raw screenshots or large perf dumps without interpretation are not sufficient.

4. Required Protocol and Test Tools

4.1 Server and Load Generator

```
./server <port> [options]
```

```
./loadgen <server-ip> <port> <clients> <requests-per-client> <workload>
```

The instructor provides loadgen. Students implement the server. A small manual client such as nc may be used for functional debugging.

4.2 Required Commands

Request	Purpose	Example Response
echo <text>	lightweight request	OK hello
time	return current server time	TIME 2026-09-11 10:30:00
compute <work>	CPU-intensive/slow work	RESULT compute finished
quit	close connection	BYE

5. Level 1 - Build a Correct Nonblocking epoll Server

5.1 Required Architecture

```
listen_fd  ----\  
client_fd1 -----\  
client_fd2 -----> epoll_wait() -> handle ready FD -> return to epoll_wait()  
client_fd3 -----/  
...        ----/  
ONE server process / ONE event-loop thread
```

5.2 Required Implementation

- Create, bind, listen, and set the listening socket to nonblocking mode.
- Create one epoll instance with `epoll_create1()` and register `listen_fd`.
- Accept all pending connections when the listening socket is ready; newly accepted client sockets must be nonblocking and registered with epoll.
- Maintain a small receive buffer and parsing state for each client. TCP is a byte stream: one `read()` is not guaranteed to contain exactly one command.
- Handle multiple complete lines already present in a client's buffer before returning to `epoll_wait()`.
- Handle EOF, malformed/oversized requests, and common `EAGAIN/EWOULDBLOCK` cases without terminating the server.

5.3 Required Functional Experiments

Demonstrate all of the following before performance measurement:

Experiment	Required Trigger	What to Explain
Multiple clients	keep several client connections active	why one event-loop thread does not require one thread per client
Partial command	split one command across two or more TCP writes	why per-client buffering is necessary
Multiple commands	send multiple newline-terminated commands in one write	why one readiness event may contain more than one request
Client disconnect	close a client normally and unexpectedly	how EOF/error removes the FD from server state

5.4 Baseline Load Test

Run the instructor-provided `loadgen` using the instructor-specified baseline workload. Record at least:

- number of concurrent clients
- total completed requests
- throughput (requests/sec)
- median or average request latency reported by `loadgen`
- server CPU observation

This Level 1 result becomes the functional/performance baseline for later architectural changes.

5.5 Required Explanation

1. What does epoll readiness mean, and what does it not guarantee?
2. Why must client sockets still be nonblocking when using epoll?
3. Why can TCP split or combine application messages independently of `read()` calls?
4. Why does a single event-loop thread avoid much of the per-client thread overhead?

6. Level 2 - Integrate timerfd/eventfd and Offload Slow Work

6.1 Unified FD Event Loop

Add two non-socket event sources to the same epoll instance:

FD	Event	Required Action
listen_fd	new connection	accept/register client
client_fd(s)	request/data ready	read/parse/respond or submit compute
timer_fd	periodic expiration	read expiration counter and print statistics
event_fd	worker completion	read eventfd and deliver completed result

6.2 timerfd Requirement

- Create a periodic timerfd using CLOCK_MONOTONIC and add it to epoll.
- Use a 5-second interval unless the instructor specifies otherwise.
- When readable, consume the expiration counter and print active clients, interval request count, throughput, and completed compute jobs.
[STATS] interval=5 sec
active_clients=124
requests=12480
throughput=2451 req/s
completed_compute=38

6.3 Two Required Compute Architectures

Implement and compare the same compute request in two versions:

Version	Architecture	Expected Effect
A - Blocking	event-loop thread performs compute directly	epoll loop cannot service unrelated ready FDs during compute
B - Offload	event loop submits compute to one worker; completion returns via eventfd	event loop remains available for lightweight requests

6.4 One-Worker Requirement

- Use one worker thread for the required Level 2 implementation.
- At most one compute job may be outstanding. If the worker is busy, returning BUSY for another compute request is acceptable.
- The worker must not directly manipulate epoll-owned client state. Store completion information and notify the event loop through eventfd.
event loop: receive compute -> submit to worker -> return to epoll_wait()
worker: perform compute -> store result -> write(eventfd)
event loop: read(eventfd) -> retrieve result -> respond to original client

6.5 Required Responsiveness Experiment

This experiment is mandatory and should use the same compute workload in Version A and Version B.

Client A: send compute <work>

Client B: while compute is running, repeatedly send time or echo requests

Measure the latency of Client B's lightweight requests in both versions and report a compact comparison table.

Architecture	Compute Running?	Light-Request Latency	Interpretation
A - blocking event loop	yes		
B - worker offload	yes		

6.6 Required Explanation

5. Why can a server that uses nonblocking sockets still become unresponsive?
6. Why is eventfd a natural worker-to-event-loop completion mechanism?
7. What problem would appear if the event loop had no reliable completion notification?
8. Why is higher total throughput not the only reason to offload compute work?

7. Level 3 - Worker Pool, Bounded Queue, Scaling, and Backpressure

7.1 Required Architecture

```
+--> worker 1 --\
epoll loop -> bounded job queue -> worker 2 ----> completion queue -> eventfd -> epoll loop
+--> ...      /
```

The event loop owns socket/epoll state.

Workers own compute execution.

- Replace the single worker with a configurable fixed worker pool, for example --workers N.
- Use a bounded job queue. Queue capacity must be configurable or clearly defined.
- Use a completion queue or equivalent safe structure so more than one completed job can be associated with one or more eventfd notifications.
- When the job queue is full, apply explicit backpressure. For the required implementation, returning BUSY is acceptable.
- Protect shared queue state correctly with appropriate synchronization primitives.

7.2 Required Scaling Experiment

Use the same compute request definition and load generator while varying worker count:

```
./server <port> --workers 1
./server <port> --workers 2
./server <port> --workers 4
./server <port> --workers 8
```

If the target has fewer CPUs, choose an instructor-approved range that includes below, near, and above the useful CPU parallelism.

For each worker count, report:

Workers	Throughput	Latency	BUSY %	CPU / task-clock	Interpretation
1					
2					
4					
8					

7.3 Required Backpressure Experiment

Choose one fixed worker count and compare at least two bounded job-queue capacities under the same offered load. The goal is to show the trade-off between queueing and rejection/backpressure.

Queue Capacity	Throughput	Latency	BUSY / Rejected	Interpretation
small				
larger				

7.4 perf Observation

For at least two meaningful configurations, collect a short perf stat observation such as:

```
perf stat -p <server-PID> \
-e task-clock,context-switches,cpu-migrations \
sleep 10
```

You may add other relevant events available on your system. Do not compare raw counters without considering measurement duration and workload. Use perf evidence to support, not replace, your latency/throughput explanation.

7.5 Required Analysis

9. Why does throughput eventually stop increasing as worker count grows?
10. Which resource or architectural point became the bottleneck in your experiment?
11. Why can adding workers increase context switches or contention?
12. What does a bounded queue protect the server from?
13. Why can a smaller queue increase BUSY responses but reduce queueing latency for accepted work?
14. Under overload, why is explicit backpressure preferable to unbounded queue growth?

8. Evidence Required

Level	Implementation Evidence	Runtime / Measurement Evidence	Analysis Evidence
1	epoll server + per-client state	partial/multiple reads + baseline load	readiness, nonblocking, TCP framing
2	timerfd + eventfd + one worker	Version A vs B responsiveness comparison	event-loop blocking + worker completion
3	worker pool + bounded queues	worker scaling + queue/backpressure + perf	saturation, bottleneck, backpressure

9. Submission Package

- Server source code and Makefile. Clearly identify the architecture/options corresponding to each completed level.
- README with exact build/run commands, protocol definition, worker/queue options, and measurement procedure.
- Technical report (recommended 5-7 pages) organized as Level 1 / Level 2 / Level 3.
- Selected functional evidence showing correct handling of partial TCP reads and multiple commands.
- Level 2 blocking-vs-offload latency comparison using the same compute workload.
- Level 3 worker-scaling and queue-capacity tables/plots, plus selected perf evidence.
- Architecture diagram for Level 2/3 showing ownership of sockets, queues, worker(s), eventfd, and timerfd.
- A short AI-use statement if AI tools were used, including at least one generated suggestion or code issue that you verified, changed, or rejected.

10. Evaluation

Category	Weight	What Is Evaluated
Implementation	35%	correct epoll/eventfd/timerfd/worker-pool behavior and error handling
Experiment & Measurement	30%	controlled load, repeatability, latency/throughput/backpressure evidence
Technical Explanation	25%	correct reasoning about event loops, blocking, synchronization, scaling, and bottlenecks
Code Quality & Report	10%	ownership, bounded state, reproducibility, clear architecture and concise evidence

Performance is not graded by absolute numbers across different hardware. The quality of the controlled comparison, the correctness of the measurement method, and the explanation of observed behavior are graded.

11. Final Check-Off Questions

- Show a command split across multiple read() calls and explain how your per-client buffer reconstructs it.
- Explain why epoll readiness does not guarantee that arbitrary application work cannot block the event loop.
- Before running the Level 2 experiment, predict how Client B latency will differ between Version A and Version B.
- Disable or bypass eventfd temporarily and explain the completion-notification problem that appears.
- Increase worker count and predict where scaling will stop on your machine before showing the measured result.
- Reduce queue capacity and predict how BUSY rate and latency will change before running the experiment.
- Point to one perf observation and explain how it supports or contradicts your bottleneck hypothesis.

Build the server. Load it. Measure what changes. Explain why.